Docent: Dirk Hostens

Apps for Android



Opdracht: Viewmodel en state in Jetpack Compose

Bronnen:

Zie Toledo

In een reeks van opdrachten maken we een app om een lijst van todo items te beheren. We zullen taken aanmaken en toekennen aan gebruikers en uiteraard ook een gebruikersbeheer voorzien. Elke opdracht zal verder bouwen op de app van de vorige opdracht.

Na deze opdracht ken je de architectuur waarbij Jetpack Compose via StateFlow en UDF werkt met state en events. Met gebruik van een ViewModel kan de state en data bewaren.

In deze opdracht

- Leer je om een correcte architectuur van je app op te zetten
- Maak je gebruik van Viewmodel, Stateflow en UDF

We passen eerst het project aan zodat we in de volgende opdracht de navigatie kunnen toevoegen zonder dat er refactoring van de code nodig is.

In eerste fase voeg je een nieuw Kotlin bestand toe ToDoApp.kt. Zorg evoor dat MainActivity the ToDoApp aanroept. ToDoApp.kt roept op zijn beurt de composable function AddEditToDoScreen aan.

```
@Composable
fun ToDoApp(…) {
    AddEditToDoScreen(…)
}
```

In ToDoApp() zullen we later de navigatie uitwerken. Plaats ToDoApp in the package ui. Je kan opteren dit door Gemini te laten doen maar het kan wellicht sneller door zelf de nodige stappen te doorlopen.

Met de introductie van App Architecture moet Gemini rekening houden met een aantal regels zoals bv. het gebruik van ViewModels. Om dit niet telkens te moeten herhalen bij het gebruik van de Gemini Agent is het interessant om een aantal 'afspraken' vast te leggen in de settings van Gemini (rules) in Android Studio. Dit is een voorbeeld van wat je in de Rules kan opnemen. Later zullen we dit uitbreiden wanneer we navigatie toevoegen of de repository layer uitwerken.

*** Voorbeeld Rules in Agent settings ***

We make use of the Kotlin language and the Jetpack Compose toolkit.
We make use of ViewModels and StateFlow. Every Viex/Screen composable has his own ViewModel and the Viewmodel is created in the constructor of the Screen composable.

De opdracht bestaat erin om alle 3 de schermen zo aan te passen zodat er correct gebruik gemaakt wordt van ViewModels.

Vergeet niet om de Viewmodel libraries toe te voegen. Standaard bij een nieuw Android-project wordt er gebruik gemaakt van een version catalog (libs.versions.toml). De build.gradle files verwijzen dan naar deze version catalog. Vraag dan ook aan Gemini om dit eerst te analyseren en eerst de libs.versions.toml aan te passen en in tweede fase de build.gradle. Vergeet niet om uw project opnieuw te synchroniseren (sync now).

In mijn oplossing werden alle variabelen uit AddEditToDoScreen verplaatst naar de UIState variabele in het ViewModel. Dit is niet helemaal correct want de isUserDropdownExpanded status is een UI-specifieke state en hoort niet thuis in de ViewModel. Dit moet dus uit de State worden verwijderd en als remberSaveable var gedefineerd worden in AddEditToDoScreen.

Veel succes!